

SafePay Change Request — Implementation Plan and Guide

Document: changerequestimplementation_guide.md
Consolidated: 19 September 2026
Project root: C:/Users/Aditya Rao/Downloads/Training/Project/SafePay
Status: Consolidated planning baseline. Documentation creation is authorized; application implementation, migration execution, and phase advancement require the user's subsequent approval.
Verified runtime baseline: User-supplied clean full-suite result: **918 tests, 0 failures, 0 errors, 0 skipped**. This is evidence from the user's run, not a test run by Codex.

Contents

- 1. Authority, provenance, and working boundaries
- 2. Final scope and corrections to earlier proposals
- 3. Existing implementation and preservation matrix
- 4. Common API, security, query, and representation contract
- 5. Sequential implementation phases
- 6. Phase 1 — SYSTEM_ADMIN account directory and details
- 7. Phase 2 — SYSTEM_ADMIN user/customer directory
- 8. Phase 3 — CUSTOMER own profile
- 9. Phase 4 — CUSTOMER transaction-history filters
- 10. Phase 5 — Above-INR-1-lakh payment categories and officer priority
- 11. Phase 6 — AUDITOR review and transaction evidence
- 12. Phase 7 — AUDITOR ledger, reconciliation, exceptions, and policies
- 13. Phase 8 — SYSTEM_ADMIN statistics and failure views
- 14. Phase 9 — Final documentation and API-testing guides
- 15. Consolidated endpoint and permission matrix
- 16. Production file manifest
- 17. Test strategy, execution gates, and expected-count discipline
- 18. Critical compatibility decisions and approval checklist
- 19. Completion and continuation checklist

1. Authority, provenance, and working boundaries

1.1 Source of this contract

This guide consolidates only the change-request discussion in this same task, beginning with the user's request to plan the capabilities in the two text files, through the latest reversal of the proposed legacy backfill.

Relevant project references:

Reference	Use
Selected capabilities	Initial positive scope; later user instructions override stale entries
Deferred capabilities	Explicit exclusions from the current implementation
Pending/history notes	Historical navigation; not blanket authorization for additional features
Updated Decision Register	Current baseline contracts except where explicitly amended by this change request
Original Decision Register	Original role/API intent, subordinate to later decisions
Original PRD	Product context; does not reinstate superseded features
Backend/database context	Earlier implementation context
Master guide contract	Final understanding/testing deliverables
Project instructions	Working rules, subject to direct user instructions

Later user instructions in this task control conflicts. This guide records the latest selections; older proposals to backfill categories, expose balances in account lists, or add administrative health reporting must not be revived.
The user's instruction to create this guide and remember it authorizes documentation and a short memory note only. It does not authorize the implementation phases described here.

1.2 Working boundaries

- Read-only file listing/search/inspection is permitted.
- Scoped direct source writes are permitted only after the user approves the immediate implementation plan and affected files. Paste-ready manual implementation is not required unless requested.
- Do not run Maven tests, start the application, execute SQL/Flyway, or change database state autonomously.
- Offline compile/test-compile checks are permitted within an approved implementation phase; they are not runtime verification.
- No downloads, installations, extensions, packages, system changes, broad refactors, or unrelated edits without explicit approval.
- Applied V1–V12 migrations remain immutable. A schema change requires a separately approved forward-only migration.
- Do not treat static SQL-file inspection as proof of the live schema or grants.
- No new tests, migrations, application configuration, or source files are created merely because their names appear here.
- SafePay remains a simulated pre-settlement control layer. No live payment-rail interception or reversal is implied.

1.3 Current baseline and stale notes

The customer account APIs are implemented and their verification gate passed. The user confirmed the individual tests, the combined **60-test** account group, and the clean **918-test** full suite.

The earlier full-suite ORA-12514 failure was resolved by the user's database/service restart. It is not an open code defect. Historical “manual verification pending” text in pending_gaps.txt is superseded by the supplied passing result.

No future test total is known until the additional tests are written and counted.

2. Final scope and corrections to earlier proposals

2.1 Selected implementation scope

Role	Selected capability
SYSTEM_ADMIN	Paginated customer-account directory; customer/minimum-current-balance/account-type filters; detail and balance reads
SYSTEM_ADMIN	Paginated user/customer search and individual user detail
SYSTEM_ADMIN	Operational statistics and detailed failure reads
CUSTOMER	Own-profile GET endpoint
CUSTOMER	Transaction-history date, state, and source-account filters
CUSTOMER	Category selection only for newly created payments strictly above INR 100000.00; conditional Others reason
RISK_OFFICER	Category filtering and category-priority sorting on the existing paginated pending-review endpoint
AUDITOR	Read-only review queue/detail and transaction/risk evidence
AUDITOR	Ledger postings/entries, two existing reconciliation views, exceptions, and policy-version reads
Final stage	OpenAPI; master backend understanding guide; comprehensive API-testing guides

“Admin” means the combined persona holding SYSTEMADMIN, RISKOFFICER, and AUDITOR. Each endpoint still checks its exact authority. A combined persona receives the union of those permissions, not a new ADMIN authority or CUSTOMER ownership.

2.2 Explicit exclusions — no implementation steps in this contract

Item	Disposition
Account disabling/reactivation	Excluded from this contract
Account deletion or direct balance-edit APIs	Prohibited
Branch/location filtering	Excluded; current account model has no location field
CUSTOMER password change/reset	Excluded
SYSTEM_ADMIN force-password-reset	Excluded
RISK_OFFICER general account directory	Excluded
New SYSTEM_ADMIN health endpoint/capability	Removed entirely from this change request; no health DTO, service, probe, scheduler-property inspection, or health-specific tests
Oracle vector/embedding/semantic-search integration	Discarded, not deferred and not a pending item ; no prototype, model, dependency, index, configuration, or future implementation slot
Unrelated suggestions appearing elsewhere in pending_gaps.txt	Not automatically included; for example, the support/contact-email form is not part of the selected change request

Existing basic application health functionality is not removed: the exclusion concerns the newly proposed SYSTEM_ADMIN health feature. Previously excluded or future product ideas—Maker-Checker, editable risk policies, safety preferences, disputes, device/location risk, general AI assistance—are not expanded or planned here.

2.3 Final decisions that replace earlier drafts

Topic	Final decision
Account-list fields	Exactly accountId, ownerId, ownerName, maskedAccountNumber, accountType, bankName, ifscCode, status
Account detail	Formatted account columns, full account number, all three balances; owner name as an additional descriptive field
Balance filtering	minCurrentBalance, inclusive currentBalance >= value; never reserved or available balance filtering
Full account number	Individual SYSTEM_ADMIN account detail only among these new staff account responses
Auditor reservation figures	Stored reservation, calculated reservation, and difference permitted; current/available balance omitted
Categories	One enum containing five values
New-payment applicability	Strictly amount > 100000.00; exactly 100000.00 remains uncategorized
Others note	Reuse purpose; required trimmed 1–140 characters for new above-threshold OTHERS payments
Existing rows	Category remains NULL; original purpose remains exactly as stored
Legacy presentation	Above-threshold category NULL displays “Not specified”; not a sixth enum
Legacy queue position	After the five real categories
Legacy lower-value responses	Category property omitted, like new lower-value payments
Seed handling	No category/purpose backfill, no reseed, no default OTHERS, no replacement with “not specified”
Review priority	MEDICAL, LOAN, FRIENDS_FAMILY, INVESTMENTS, OTHERS, NULL; oldest first within each group
Documentation	Final stage after implementation and verification; OpenAPI before frontend integration

3. Existing implementation and preservation matrix

Existing component	Current behavior	Preservation rule
AccountController / AccountService	Customer-owned list/detail/balance	Keep existing routes, owner checks, and customer DTOs
AccountDao / Account	Ownership reads, financial fields, locks	Add staff read queries only; no account mutation
AdminUserSecurityController / service	User status, roles, session revocation	Add GET operations; preserve mutation and audit semantics
TransactionController / service	Create, authorize, list, detail, cancel, risk explanation	Extend only selected query/create metadata paths
RiskReviewController / service	Pending queue, detail, decisions, notes	Extend queue filters/order; preserve financial decisions
AuditController / AuditQueryService	Global audit and role-scoped timelines	Keep intact; new evidence reads use separate routes
Ledger/exception/policy entities and repositories	Financial evidence and internal reads	Add non-locking read queries only
PagedResponse	items/page/size/totalElements/totalPages/first/last	Reuse unchanged
GlobalExceptionHandler	Existing problem responses	Reuse existing validation/access/not-found handling
SecurityConfig / JWT converter	Exact authorities, live user/role/security-version checks	Add explicit routes; preserve authentication mechanics
Schedulers, OTP, idempotency engine, settlement/state/funds services	Canonical processing	No algorithm change; only create-request fingerprint input gains category where applicable
Existing SQL views and grants	Reporting foundations	Read existing structures; do not rewrite them

Payment category must not become a risk signal. Amount-only classification, OTP, review eligibility, protection periods, reservation, cancellation, settlement, ledger entries, and audit immutability retain their contracts.

4. Common API, security, query, and representation contract

4.1 Types and validation

Concern	Contract
Prefix	/api/v1
IDs	Long internally; string identifiers in DTOs following current conventions
Money	BigDecimal internally, never binary floating point; new financial fields use exact two-decimal strings
Existing DTO money	Preserve current types/serialization; do not globally refactor existing BigDecimal responses
Dates	OffsetDateTime; normalize query timestamps to UTC
New date ranges	from inclusive, to exclusive; reject from >= to when both supplied
List defaults	page=0, size=20; validate page >= 0 and 1 <= size <= 100
Counts	Database count query with identical filter predicates
Ordering	Fixed order plus unique ID tie-breaker; no unrestricted client property sorting
No results	Empty page, not an error
Invalid input	Existing 400 response conventions
Missing detail	Existing resource-not-found convention
Authorization	Existing 401/403 conventions; no entity/data query before necessary authorization

New date filtering uses **from** <= **timestamp** < **to**. Do not change existing audit endpoint date semantics as collateral work. Full-day UI selections should be converted to start-of-day and next-day-start in an explicit timezone.

Do not reuse MoneyUtility.requireValidTransactionAmount() for minCurrentBalance: it enforces a minimum INR 1 transaction amount. A minimum-balance filter of zero is valid.

4.2 Query methods and transactional boundaries

- New staff read methods use read-only transactions and exact-role checks.
- Add methods to existing restricted Repository interfaces rather than replacing them with broad CRUD repositories.
- Queries remain parameterized. Escape wildcard search input deliberately.
- Fetch to-one associations needed for DTOs; do not paginate a to-many fetch join.
- For paginated user lists, fetch role assignments for the selected page in one batch.
- All filtering/ordering is applied before database pagination. Do not retrieve everything and filter in Java.
- Do not take write locks for reporting.
- Explicit JDBC projections over views avoid additional JPA entities.
- Reuse existing DTO mapping and SensitiveDataMasker where compatible.
- Use bound parameters and fixed allowlisted identifiers for native SQL.
- Existing grants are checked before implementation execution; missing live grants require an explicit forward-only correction plan, not broad privilege escalation.
- Financial/report data changes concurrently. Deterministic pagination is not a frozen snapshot across separate requests.

4.3 Shared staff access helper

New file: **StaffReadAccess.java** in the manifest, StaffReadAccess.java.

Method: requireActiveRole(actorUserId, requiredRole).

Use UserDao and UserRoleDao to confirm an active user with the required authority. New account, auditor-evidence, and operations services use it. Existing administrator/audit services retain their existing equivalent checks; no wholesale refactor.

4.4 Route protection

Add explicit root and descendant matching as appropriate:

Route	Authority
/api/v1/admin/accounts and children	SYSTEM_ADMIN
/api/v1/admin/users and children	SYSTEM_ADMIN
/api/v1/admin/operations and children	SYSTEM_ADMIN
/api/v1/users/me	CUSTOMER
/api/v1/audit and children	AUDITOR
Existing /api/v1/admin/risk-reviews routes	RISK_OFFICER, unchanged

Apply controller method security as well. Auditor routes do not broaden the existing officer controller annotation. SYSTEM_ADMIN alone does not gain approval or general auditor-evidence rights.

5. Sequential implementation phases

The initial recommended sequence is preserved, with categories inserted before the new auditor DTO consumers to reduce repeated integration changes.

Phase	Deliverable	Gate
0	Approve this contract and resolve compatibility choices	No implementation until approved
1	SYSTEM_ADMIN account reads	Focused/security/query verification
2	SYSTEM_ADMIN user/customer directory	Query counts and security verification
3	CUSTOMER own profile	Identity and privacy verification
4	CUSTOMER transaction filters	Ownership and date-boundary verification
5A	Approve category migration/deployment and replay compatibility	Separate database execution authorization
5B	Category persistence/create validation/response mapping	Legacy and amount-boundary verification
5C	Officer category filtering/priority	Global pagination and decision-regression verification
6	AUDITOR review and transaction evidence	Read/action separation verification
7A	Ledger reads	Financial evidence mapping verification
7B	Reconciliation reads	Oracle projection and privacy verification
7C	Exception and policy reads	Safe fields and historical evidence verification
8	SYSTEM_ADMIN stats and failures	Counter/query/role verification
9	OpenAPI and final guides	Document only verified final behavior

At every gate: explain the exact affected files, implement only the approved increment, run permitted compilation, provide user-run focused test instructions/counts, inspect results, and obtain confirmation before advancing. No retrospective claim that the entire contract was implemented merely because one phase passed.

6. Phase 1 — SYSTEM_ADMIN account directory and details

6.1 Endpoints

Method	Endpoint	Parameters/return
GET	/api/v1/admin/accounts	customerId, minCurrentBalance, accountType, page, size; PagedResponse
GET	/api/v1/admin/accounts/{accountId}	AdminAccountDetailResponse
GET	/api/v1/admin/accounts/{accountId}/balance	Existing AccountBalanceResponse

Only customer-owned SAVINGS/CURRENT accounts are included. Internal clearing/control accounts are excluded from these routes. Inactive customer accounts remain inspectable.

6.2 Exact fields

List item — only these fields	Detail — formatted account columns
accountId	accountId
ownerId	ownerId
ownerName	ownerName, additional descriptive field
maskedAccountNumber	accountNumber, full
accountType	accountType
bankName	bankName
ifscCode	ifscCode
status	status
—	currency
—	currentBalance
—	reservedAmount
—	availableBalance
—	versionNo
—	createdAt
—	updatedAt

List items contain **no balances, currency, timestamps, or full account number**. The normal PagedResponse envelope is retained. The balance endpoint reuses AccountBalanceResponse and keeps its maskedAccountNumber. No raw User entity, password data, or nested entity graph is exposed.

Illustrative list item:

```
{
  "accountId": "501",
  "ownerId": "101",
  "ownerName": "Example Customer",
  "maskedAccountNumber": "*****9012",
  "accountType": "SAVINGS",
  "bankName": "Example Bank",
  "ifscCode": "ABCD0123456",
  "status": "ACTIVE"
}
```

6.3 Methodology and methods

1. Validate SYSTEM_ADMIN through the shared helper.
2. Validate positive customerId and permitted accountType.
3. Validate minCurrentBalance >= 0, at most two decimal places, NUMBER(18,2) range.
4. Combine supplied filters using AND.
5. Apply currentBalance >= minCurrentBalance.
6. Sort accountId ASC.
7. Map list/detail independently so full numbers cannot enter list serialization.

Service methods:

- searchAccounts(administratorId, customerId, minCurrentBalance, accountType, page, size)
- getAccount(administratorId, accountId)
- getBalance(administratorId, accountId)

AccountDao additions:

- searchCustomerAccounts(customerId, minCurrentBalance, accountType, pageable)
- findCustomerAccountById(accountId)

Both queries enforce account type/ownership eligibility. Fetch owner through a to-one association where needed.

6.4 Files

Create NO2–NO6; extend E01/E02; reuse AccountBalanceResponse, SensitiveDataMasker, PagedResponse and the existing entity. Do not change AccountService's customer methods or any account mutation method.

6.5 Acceptance checks

Verify exact list-field allowlist; full number only in administrator detail; balances only in permitted responses; type/customer/minimum filters in combination; zero and precision limits; excluded internal accounts; inactive accounts; stable pagination/counts; all single-role denials and combined-admin success.

7. Phase 2 — SYSTEM_ADMIN user/customer directory

7.1 Endpoints

Method	Endpoint	Contract
GET	/api/v1/admin/users	q, role, status, page, size
GET	/api/v1/admin/users/{userId}	Individual user security/identity read

Reuse AdminUserSecurityResponse and PagedResponse. role=CUSTOMER supplies a customer picker for account filtering.

7.2 Methodology

1. Add read-only service methods alongside existing administration methods.
2. Use existing active-administrator checking without lockParticipants().
3. Search name/email case-insensitively and mobile text using a bounded q.
4. Apply RoleName and UserStatus filters with AND semantics.
5. Use EXISTS for the role predicate, preserving one row per user.
6. Sort userId ASC.
7. Batch-load role assignments for the selected user IDs, including role entities needed for mapping.
8. Return existing safe user DTO fields; never serialize User.

Methods:

- searchUsers(administratorId, q, role, status, page, size)
- getUserDetails(administratorId, targetUserId)
- UserDao.searchUsers(q, role, status, pageable)
- UserRoleDao.findAllForUsers(userIds)

7.3 Files and tests

Extend E03–E07; no new production DTO/service/controller needed.

Extend existing AdminUserSecurityControllerTest, AdminUserSecurityServiceImplTest and repository integration coverage. Verify multi-role users are not duplicated, page totals are accurate, CUSTOMER filters work, and all existing status/role/session mutation tests still pass.

8. Phase 3 — CUSTOMER own profile

8.1 Endpoint and fields

GET /api/v1/users/me

Response: userId, fullName, email, mobileNumber, status.

The user ID comes solely from AuthenticatedUser. No target ID/role parameter is accepted. Existing authentication already supplies role information.

8.2 Methodology

1. Require CUSTOMER at the route/controller.
2. Call UserService.getOwnProfile(customerUserId).
3. Reuse UserDao.findById().
4. Map to CustomerProfileResponse.
5. Omit password hashes, security version, failure counters, secrets and tokens.

8.3 Files and tests

Create No7/No8; extend Eo1/Eo8/Eo9. Reuse existing User entity and DAO.

Test principal-derived identity, denial for staff-only identities, allowed profile fields, missing-resource handling and absence of sensitive fields. Do not add a general public GET /users/{id} as a side effect.

9. Phase 4 — CUSTOMER transaction-history filters

9.1 Existing endpoint extension

GET /api/v1/transactions

Parameter	Meaning
from	Inclusive creation timestamp
to	Exclusive creation timestamp
state	Existing TransactionState
sourceAccountId	Source account filter within the customer's owned payments
page / size	Existing pagination

Keep current TransactionSummaryResponse and ordering: createdAt DESC, transactionId DESC.

9.2 Methodology

1. Preserve the original no-filter behavior.
2. Derive customer identity from the authenticated principal.
3. Validate range, enums, ID and pagination.
4. Add a filtered listTransactions overload.
5. Use ownership AND every supplied filter in the database.
6. A foreign sourceAccountId yields no owned matches, not cross-owner data.
7. Keep count-query predicates aligned.

Methods:

- Existing listTransactions(customerUserId, page, size) remains.
- New listTransactions(customerUserId, from, to, state, sourceAccountId, page, size).
- TransactionDao.searchOwned(customerUserId, from, to, state, sourceAccountId, pageable).
- Keep findAllOwned() for the original no-filter path.

9.3 Files and tests

Extend E10–E13. No new production file required for filtering itself.

Extend TransactionControllerTest, TransactionQueryServiceTest, TransactionDaoIntegrationTest and relevant Oracle service coverage. Verify offsets, exact time boundaries, combined filters, empty pages, ownership and unchanged unfiltered calls.

10. Phase 5 — Above-INR-1-lakh categories and officer priority

10.1 Applicability and invariant

One new enum: PaymentCategory.

Value	Display label	Priority
MEDICAL	Medical	1
LOAN	Loan	2
FRIENDS_FAMILY	Friends & Family	3
INVESTMENTS	Investments	4
OTHERS	Others	5
NULL, not an enum	Not specified, historical above-threshold rows only	Last

The threshold is **amount > 100000.00**, using BigDecimal. Exactly INR 100000.00 is HIGH and requires no category; INR 100000.01 is above threshold.

The category threshold is metadata validation tied to the current approved band boundary, not a second risk engine. Centralize the comparison and verify it against current policy boundaries in tests. Never use float/double.

MEDICAL is a customer-declared category, not verified medical urgency. No additional emergency flag or evidence workflow is introduced.

10.2 Final legacy table — no backfill

Payment group	Database category	API/UI behavior	Purpose
Existing <= INR 100000.00	NULL	Category property omitted; no category UI	Preserve original value, including null
Existing > INR 100000.00	NULL	category may be null; UI shows “Not specified” where applicable	Preserve original value, including null
New <= INR 100000.00	NULL	No category selection/property	Existing optional purpose, max 280
New > INR 100000.00, non-OTHERS	Selected category	Required at creation and displayed	Optional purpose, max 280
New > INR 100000.00, OTHERS	OTHERS	Required at creation and displayed	Required trimmed reason, 1–140

All existing records, including all V12 seeds and runtime-created records predating this change, remain uncategorized. Do not infer MEDICAL from “Hospital advance” or infer any other category from free text.

Do not UPDATE old rows to OTHERS, rewrite purpose, use a default category, add a sixth enum, or reseed. Old above-threshold payments remain authorizable/reviewable/settleable under existing lifecycle rules.

10.3 Schema plan and execution gate

Proposed migration name: V13__payment_category.sql **only if V13 remains the next available version at implementation time.**

Location is listed as Mo1 in the manifest. V12 is in the showcase location; review all configured migration locations before selecting the next version.

Proposed column: PAYMENTTRANSACTION.PAYMENTCATEGORY VARCHAR2(20 CHAR), nullable, no non-null default.

Constraint intent:

- 1. Non-null values must belong to the five-value set.
- 2. Payments at or below INR 100000.00 cannot store a non-null category.
- 3. OTHERS requires a nonblank trimmed purpose within 140 characters.
- 4. Above-threshold NULL must remain permitted for historical records.

Because historical and new above-threshold rows share the table, a universal NOT NULL requirement is invalid. The create request/service/factory enforces mandatory category for **new** above-threshold payments. The database cannot distinguish creation era without adding further metadata, which is not planned.

Approval and deployment sequence:

- 1. Review migration DDL and exact new constraints.
- 2. User approves and performs or explicitly authorizes execution.
- 3. Verify existing row counts, balances, states and purpose values remain unchanged.
- 4. Verify new column is null on all existing rows and Hibernate mapping matches.
- 5. Coordinate application version and migration target before startup/integration tests.
- 6. If test/run configurations still pin SPRINGFLYWAYTARGET=12, obtain approval to advance to the actual new version. Keeping target 12 would leave the new entity mapping without its column.
- 7. Keep both approved migration locations where the existing showcase database requires them.
- 8. Do not rebuild the schema, run Flyway clean/repair, edit old migrations, or automatically change deployment configuration.

Adding one field is the sole planned schema change for this contract. Any additional index or privilege change requires evidence and separate review.

10.4 Creation request rules

POST /api/v1/transactions retains existing fields and gains conditional category.

```
{
  "sourceAccountId": 501,
  "beneficiaryId": 701,
  "amount": 5000.00,
  "purpose": "Monthly expenses",
  "customerReference": "PAY-001"
}
```

```
{
  "sourceAccountId": 501,
  "beneficiaryId": 701,
  "amount": 150000.00,
  "category": "MEDICAL",
  "purpose": "Hospital advance",
  "customerReference": "PAY-002"
}
```

```
{
  "sourceAccountId": 501,
  "beneficiaryId": 701,
  "amount": 150000.00,
  "category": "OTHERS",
  "purpose": "Equipment purchase",
  "customerReference": "PAY-003"
}
```

- At/below threshold: omit category or send null; reject any non-null category.
- Above threshold: category is mandatory and must be one of the enum values.
- OTHERS above threshold: trim purpose and require 1–140 characters.
- Other categories: preserve optional purpose and its existing 280-character bound.
- No new otherReason property/column.
- Blank input normalization must happen before conditional validation.
- Validate amount precision/minimum using existing rules before category applicability.
- Do not add unconditional @NotNull to category.
- The UI clears category when the amount moves below the threshold; normal purpose text remains available.
- Authorization, cancel, OTP and review-decision request bodies do not gain category.

10.5 Persistence, DTOs, and idempotency

- Add EnumType.STRING mapping and getter in TransactionDb; do not store enum ordinals.
- Set category at instruction creation; do not add an edit-category endpoint.
- Extend creation factory/service narrowly. Retain old Java signatures where safe, but do not use overloads to bypass mandatory new-payment validation or fabricate OTHERS.
- Update affected test constructors/fixtures deliberately.
- Add category to TransactionResponse, TransactionSummaryResponse and RiskReviewSummaryResponse; the existing review-detail wrapper inherits it.
- Include purpose in review summary/detail so the officer can inspect the Others reason.
- Legacy null is valid in response mapping. Do not make DTO construction reject it.
- Lower-value responses omit category entirely; above-threshold legacy responses can carry category:null.
- Use property-specific serialization/mapping, not a global Jackson null-omission change. Test mixed pages and avoid inadvertently hiding existing unrelated fields.
- Verify cached old JSON responses still deserialize after DTO evolution.
- Extend TransactionController.canonicalCreateRequest() to include category when supplied, preserving the old canonical map for lower-value requests.
- Same key plus changed category is an idempotency conflict, as with other changed semantic inputs.
- Do not rewrite stored idempotency fingerprints or response bodies.

Compatibility gate: an old above-threshold create request without category will fail the new request validation before replay lookup. Existing stored payments still function, but this is different from replaying an old POST. Before Phase 5 implementation, explicitly acknowledge that behavior or separately approve narrowly scoped replay compatibility. Do not claim old high-value POST replay works automatically.

10.6 Officer queue

Extend existing GET /api/v1/admin/risk-reviews:

Parameter	Proposed behavior
category	Optional enum filter; omitted includes legacy NULL
sort	PRIORITY default; OLDEST retains FIFO option
page / size	Existing defaults/bounds

Sort PRIORITY globally in SQL:

MEDICAL -> LOAN -> FRIENDS_FAMILY -> INVESTMENTS -> OTHERS -> NULL, then requestedAt ASC, approvalId ASC.

Only eligible pending reviews are returned. Existing workflows create those reviews after the required verification; category does not create a review for a small payment or skip OTP.

Methods:

- Keep listPending(officerId, page, size).
- Add listPending(officerId, category, sort, page, size).
- Add a filtered pending-review DAO query with matching count.
- Use allowlisted sort values; no client-controlled SQL/property names.
- If priority CASE ranks repeat enum order in SQL, test their consistency.
- Keep review decisions, state guards, note mutation, reservation release and settlement processing unchanged.

No separate API or frontend page is necessary. Existing category-null high-value reviews remain usable and appear after categorized rows in PRIORITY mode. OLDEST avoids imposing category priority when the officer deliberately selects FIFO; no new aging algorithm is introduced.

10.7 Files

Create No9/Mo1. Extend E10/E12/E14–E21 and relevant tests. No payment-category table, category management controller, separate note field, new role, or weighted-risk configuration is required.

10.8 Tests

Input/condition	Expected
100000.00, category absent	Accepted subject to existing validation
100000.00, category MEDICAL	400
100000.01, category absent	400 for new creation
100000.01, category MEDICAL	Accepted subject to existing validation
100000.01, OTHERS + blank reason	400
OTHERS reason length 140	Accepted
OTHERS reason length 141	400
Existing high-value null category	Existing lifecycle operations continue
Existing purpose text	Preserved exactly
Mixed response page	Category absent on lower rows; correct high-row representation
Priority spans multiple pages	Global order/count correct; stable tie-breakers
Same create key, different category	Existing conflict behavior
Old persisted response JSON	Deserializes without invented category
Existing financial/OTP/state tests	Remain correct

10.9 Frontend boundary

No active frontend package was found during the planning inspection. Do not invent an existing view-model path or start frontend scaffolding.

Record only the integration requirements for the later frontend work: conditional selector, conditional Others reason, “Not specified” for historical high-value records, queue category filter and sort control. The API-testing guide remains a final-stage deliverable.

11. Phase 6 — AUDITOR review and transaction evidence

11.1 Shared auditor architecture

Create N10–N12: AuditEvidenceController, AuditEvidenceService, AuditEvidenceServiceImpl.

Use `/api/v1/audit` routes with AUDITOR checks and the shared access helper. Preserve existing `AuditController/AuditQueryService` timeline and global-audit methods.

11.2 Read-only review routes

Method	Endpoint	Contract
GET	<code>/api/v1/audit/risk-reviews</code>	status=PENDING, page, size; select another valid status for historical reviews
GET	<code>/api/v1/audit/risk-reviews/{reviewId}</code>	Existing detail response

Methods: `listReviews(auditorId, status, page, size)`, `getReview(auditorId, reviewId)`.

Reuse `RiskReviewDao.findAllByStatusOrderByRequestedAtAscApprovalIdAsc()` and `findByApprovalId()`. Reuse review summary/detail DTOs, including their final category fields. This resolves the earlier permission discrepancy without granting auditors officer actions.

Do not automatically impose officer-specific category priority on historical auditor lists. Default auditor ordering remains the documented existing query order; additional auditor filters require scoped agreement.

11.3 Transaction and risk evidence routes

Method	Endpoint	Contract
GET	<code>/api/v1/audit/transactions</code>	customerId, state, from, to, page, size
GET	<code>/api/v1/audit/transactions/{transactionId}</code>	Payment detail plus stored risk evidence

Methods: `searchTransactions(...)`, `getTransactionEvidence(auditorId, transactionId)`.

- Add dedicated non-owner-scoped auditor read queries after authority checking.
- Never call customer services with a substituted customer identity.
- Reuse `TransactionSummaryResponse` for the list.
- New N13 composes `customerId`, existing `TransactionResponse`, and a nested typed risk-evidence list.
- Use stored factor/band/tier/explanation/evaluation information.
- Reuse existing persisted-evidence consistency validation where applicable.
- Unassessed CREATED payments have an empty risk-evidence list, not a fabricated assessment.
- Existing transaction audit endpoint remains the timeline source; no duplicate timeline route is needed.

Extend E13 and E22: `searchForAudit(...)`, `findForAuditById(...)`, `findAllForAuditByTransactionId(...)`.

11.4 Verification

Test single-role access, combined persona, denied officer mutations for auditor-only users, review histories, historical policies, unassessed transactions, missing/inconsistent evidence, and no current-account balance leakage from nested entities.

12. Phase 7 — AUDITOR ledger, reconciliation, exceptions, and policies

12.1 Ledger posting/entry reads

Method	Endpoint	Contract
GET	<code>/api/v1/audit/ledger-postings</code>	transactionId, posting status, creation from/to, page, size
GET	<code>/api/v1/audit/ledger-postings/{postingId}</code>	Header and ordered entries

Create N14, `AuditLedgerPostingResponse`, with nested header/entry records. List returns headers; detail contains entries. Add `searchForAudit(...)` and `findByPostingId(...)` to E23.

Reuse `LedgerEntryDao.findAllByPosting_PostingIdOrderByLineNumberAsc()`. Include opening-balance postings with null `transactionId`. Never assume every posting is a payment settlement.

Expose permitted posting identifiers, type/status, amount/currency, entry counts/timestamps, account IDs/masked identifiers, line numbers and debit/credit amounts. Do not expose replay keys merely because entities contain them. No edit/repair operations.

12.2 Reconciliation

Method	Endpoint	Parameters
GET	<code>/api/v1/audit/reconciliation/ledger</code>	postingId, reconciliationStatus, page, size
GET	<code>/api/v1/audit/reconciliation/reservations</code>	accountId, reconciliationStatus, page, size

Read existing `SAFEPAYOWNER.VWLEDGERRECONCILIATION` and `SAFEPAYOWNER.VWRESERVATIONRECONCILIATION`. Existing grants source includes `SELECT` for `SAFEPAY_APP`.

Create N15 `ReportingReadRepository` using existing Spring JDBC infrastructure and explicit column projections. Create N16 `AuditReconciliationResponse` with separate nested ledger/reservation records.

View	Permitted fields
Ledger	Posting identifiers/status/amount/currency, expected/actual entries, debit/credit counts/totals, reconciliation status, appropriate timestamps
Reservations	Account ID, masked number, currency, stored reservation, calculated reservation, difference, reconciliation status, updated time

Omit *currentbalance* and *availablebalance* from auditor reservation SQL projection and DTO, despite their presence in the view. Use view-derived statuses: ledger `INCOMPLETE/BALANCED/MISMATCH`; reservations `MATCH/MISMATCH`. Validate each endpoint's own allowed statuses.

These views verify balanced postings and reservation consistency. They do not provide complete ledger-to-current-account-balance reconstruction. Do not label them as such.

Native-query implementation must verify Oracle `NUMBER` and timestamp mappings, identical count filters, deterministic pagination, and no modification of underlying views.

12.3 Exception reads

Method	Endpoint	Contract
GET	<code>/api/v1/audit/exceptions</code>	transactionId, processingStage, status, occurrence from/to, page, size
GET	<code>/api/v1/audit/exceptions/{exceptionId}</code>	Safe exception evidence

Use `lastOccurredAt` for the proposed occurrence range and sort `lastOccurredAt DESC`, `transactionExceptionId DESC`; confirm these names in the phase contract.

Create N17 `AuditExceptionResponse`. Extend E24 with `searchForAudit(...)` and `findByTransactionExceptionId(...)`.

Fields include IDs/reference, transaction/posting IDs, stage, `errorCode`, `retryability/count`, next retry time, occurrence/resolution timestamps, permitted resolving-user ID and correlation ID. A controlled display explanation may be derived from known error codes.

Do not return raw `errorMessage`, arbitrary stack traces, or unrestricted resolution text by default. Unknown codes receive a generic display explanation. Do not misuse the existing audit-JSON sanitizer as a general-purpose free-text sanitizer.

12.4 Policy-version reads

Method	Endpoint	Contract
GET	<code>/api/v1/audit/risk-policies</code>	Optional existing policy status, page, size
GET	<code>/api/v1/audit/risk-policies/{policyVersion}</code>	Stored header and bands

Create N18 `AuditRiskPolicyResponse` with nested band records. Extend E25 for paginated policy listing; reuse `findByPolicyVersion()` and `RiskPolicyBandDao.findAllForPolicy()`.

Include stored version/name/status/algorithm/currency/effective dates and ordered bands with min/max amount, tier, protection code/release mode/duration and approved OTP/review/cancellation flags.

Read inactive/historical versions as stored. Do not route historical inspection through current eligible-policy selection or recalculate old payment decisions.

12.5 Verification

Cover zero-result pages, opening balances, null transaction/posting references, exact money, allowed reconciliation statuses, omission of current/available balance, historical policies/open-ended top bands, safe exception fields and authorization. No mutation of ledger, exception resolution, risk policy, or account balance is added.

13. Phase 8 — SYSTEM_ADMIN statistics and failures

13.1 Endpoints

Method	Endpoint	Contract
GET	<code>/api/v1/admin/operations/stats</code>	Current operational counters
GET	<code>/api/v1/admin/operations/failures</code>	Source, transactionId, occurrence from/to, page, size

There is **no proposed /health route**. No `Health` nested DTO, probe, liveness field, scheduler-properties dependency or new health configuration is to be created.

Create N19–N23: controller, service interface/implementation, OperationalDashboardResponse for statistics only, and OperationalFailureResponse. Reuse N15 reporting repository.

13.2 Statistics methodology

Recommended initial counters:

- Current payment counts by state.
- Current pending-review count.
- Exception counts by OPEN, RETRYPENDING, MANUALREVIEW.
- Notification counts by PENDING, RETRY_PENDING, FAILED.
- Expired protected-payment backlog.
- Settlement work currently due under the existing latest-exception/retry rules.

Use grouped/count queries, not entity loading or the existing batch-limited worker selection methods as total counts.

Where consistency across counters matters, use one Oracle SELECT with aggregate subqueries and database observation time. Document that these are current-state/workload counters, not all-time event totals or scheduler-health conclusions.

Due-work predicates must match current worker eligibility, including latest relevant settlement exception and retry time. Reporting does not invoke workers.

Method: getStatistics(administratorId).

13.3 Failure methodology

One paginated view covers transaction-processing exceptions and failed/retrying notification records.

- source is an allowlisted TRANSACTION or NOTIFICATION value; omission can include both.
- Each result includes source type and record ID to disambiguate identical numeric IDs.
- Use a defined occurrence timestamp: exception lastOccurredAt, notification failedAt where populated, otherwise updatedAt for retry records.
- Sort event time DESC with source and ID tie-breakers.
- Project code, safe display explanation, stage/status where applicable, retry information, relevant timestamps, transaction ID and correlation ID.
- Do not include notification body, personal recipient data, raw technical exception text, credentials or mail configuration.
- Database pagination/count must cover the same combined sources.
- No extra detail route is necessary if all permitted failure details fit this response.

Method: searchFailures(administratorId, source, transactionId, from, to, page, size).

13.4 Verification

Validate counters against known fixtures; no overcount from joins; correct latest-exception eligibility; notification retries/terminal failures; combined-source pagination/counts; empty state; authority denial; no SQL writes or worker invocation.

14. Phase 9 — Final documentation and API-testing guides

Only after implemented phases and their required runtime gates pass:

1. Reconcile the final endpoint/field/error/security contract.
2. Produce OpenAPI documentation, with any dependency or configuration addition separately scoped and approved.
3. Produce the master backend understanding guide.
4. Produce comprehensive role-wise API-testing guides with environment setup, request examples, expected responses, sequencing and cleanup boundaries.
5. Record what was actually verified, using user-supplied runtime evidence.
6. Complete OpenAPI before frontend integration.

Focused verification instructions during implementation are still necessary; deferring the comprehensive guides does not defer all testing.

15. Consolidated endpoint and permission matrix

All additions below are GET/read-only except the existing payment-create request gaining category metadata.

Endpoint/change	CUSTOMER	SYSTEM_ADMIN	RISK_OFFICER	AUDITOR
GET /api/v1/admin/accounts	—	Yes	—	—
GET /api/v1/admin/accounts/{accountId}	—	Yes	—	—
GET /api/v1/admin/accounts/{accountId}/balance	—	Yes	—	—
GET /api/v1/admin/users	—	Yes	—	—
GET /api/v1/admin/users/{userId}	—	Yes	—	—
GET /api/v1/users/me	Own	—	—	—
GET /api/v1/transactions filters	Own	—	—	—
POST /api/v1/transactions conditional category	Own	—	—	—
GET /api/v1/admin/risk-reviews category/priority	—	—	Yes	—
GET /api/v1/audit/risk-reviews	—	—	—	Yes
GET /api/v1/audit/risk-reviews/{reviewId}	—	—	—	Yes
GET /api/v1/audit/transactions	—	—	—	Yes
GET /api/v1/audit/transactions/{transactionId}	—	—	—	Yes
GET /api/v1/audit/ledger-postings	—	—	—	Yes
GET /api/v1/audit/ledger-postings/{postingId}	—	—	—	Yes
GET /api/v1/audit/reconciliation/ledger	—	—	—	Yes
GET /api/v1/audit/reconciliation/reservations	—	—	—	Yes
GET /api/v1/audit/exceptions	—	—	—	Yes
GET /api/v1/audit/exceptions/{exceptionId}	—	—	—	Yes
GET /api/v1/audit/risk-policies	—	—	—	Yes
GET /api/v1/audit/risk-policies/{policyVersion}	—	—	—	Yes
GET /api/v1/admin/operations/stats	—	Yes	—	—
GET /api/v1/admin/operations/failures	—	Yes	—	—

A combined three-role administrator gets the union of the three staff columns. It does not gain CUSTOMER routes without CUSTOMER itself and applicable ownership.

Existing audit timelines, auth, notifications, beneficiary, review decisions and administration mutations retain their established permissions.

16. Production file manifest

Names below are planning targets, not files already created. Existing file edits are additive/scoped unless explicitly described otherwise. Refer to phase acceptance checks before editing.

16.1 Proposed new source files

ID	Absolute path	Responsibility
N01	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/security/StaffReadAccess.java	Active exact-role helper for new staff reads
N02	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/controller/AdminAccountController.java	Three account GET routes
N03	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AdminAccountService.java	Account read contract
N04	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AdminAccountServiceImpl.java	Validation, authority, account reads
N05	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/admin/AdminAccountSummaryResponse.java	Exact eight-field list item
N06	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/admin/AdminAccountDetailResponse.java	Full permitted account detail
N07	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/controller/CustomerProfileController.java	Own-profile route
N08	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/user/CustomerProfileResponse.java	Safe profile projection
N09	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/beans/PaymentCategory.java	Five values and shared applicability semantics
N10	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/controller/AuditEvidenceController.java	Auditor evidence routes
N11	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AuditEvidenceService.java	Evidence read contract
N12	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AuditEvidenceServiceImpl.java	Authorized evidence composition
N13	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/audit/AuditTransactionDetailResponse.java	Existing transaction DTO plus typed risk evidence
N14	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/audit/AuditLedgerPostingResponse.java	Header/detail/entry records
N15	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/ReportingReadRepository.java	Parameterized JDBC projections/aggregates
N16	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/audit/AuditReconciliationResponse.java	Distinct ledger/reservation nested records
N17	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/audit/AuditExceptionResponse.java	Safe exception evidence
N18	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/audit/AuditRiskPolicyResponse.java	Stored policy and nested bands
N19	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/controller/AdminOperationsController.java	Statistics/failures only
N20	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AdminOperationsService.java	Statistics/failure read contract
N21	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AdminOperationsServiceImpl.java	Authorized operational reads
N22	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/admin/OperationalDashboardResponse.java	Statistics only; no health record
N23	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/admin/OperationalFailureResponse.java	Typed combined failure source

Small response-only child types should be nested rather than split into many tiny files. No generic framework or mapper dependency is planned. If implementation reveals a genuinely necessary extra file, explain its purpose and update the scoped plan before creating it.

16.2 Existing source files to extend

ID	Absolute path	Permitted change
E01	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/security/SecurityConfig.java	Exact new route matchers
E02	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/AccountDao.java	Staff customer-account reads
E03	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/controller/AdminUserSecurityController.java	Two user-directory GET routes
E04	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AdminUserSecurityService.java	Read method declarations
E05	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/AdminUserSecurityServiceImpl.java	Read-only user-directory logic
E06	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/UserDao.java	Paginated filtered users
E07	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/UserRoleDao.java	Page-scoped bulk role read
E08	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/UserService.java	getOwnProfile declaration
E09	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/UserServiceImpl.java	Own-profile mapping
E10	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/controller/TransactionController.java	History parameters and category fingerprint
E11	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/TransactionService.java	Filtered history overload
E12	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/TransactionServiceImpl.java	Filtered reads and creation metadata
E13	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/TransactionDao.java	Owned filters and auditor read queries
E14	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/beans/TransactionDb.java	Nullable category mapping/factory/getter
E15	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/transaction/CreateTransactionRequest.java	Conditional category/purpose validation
E16	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/transaction/TransactionResponse.java	Conditional category response
E17	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/transaction/TransactionSummaryResponse.java	Conditional category response
E18	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/dto/riskreview/RiskReviewSummaryResponse.java	Category and purpose
E19	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/controller/RiskReviewController.java	Queue parameters
E20	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/RiskReviewService.java	Queue overload
E21	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/services/RiskReviewServiceImpl.java and C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/RiskReviewDao.java	Queue validation/filter/ordering
E22	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/TransactionRiskFactorDao.java	Auditor evidence read
E23	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/LedgerPostingDao.java	Non-locking detail/search
E24	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/TransactionExceptionDao.java	Non-locking detail/search
E25	C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/java/com/ofss/repository/RiskPolicyDao.java	Paginated historical listing

M01: C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/main/resources/db/migration/V13__payment_category.sql — proposed next version only, subject to inventory and explicit migration approval.

Tests and approved final documentation are additional phase-specific files. This manifest does not authorize arbitrary editing of neighbouring files.

16.3 Reuse without planned alteration

Account, AccountStatus, AccountType, AccountService/implementation, AccountBalanceResponse, AccountSummaryResponse, AdminUserSecurityResponse, PagedResponse, SensitiveDataMasker, MoneyUtility transaction validation, LedgerEntryDao, RiskPolicyBandDao, RiskReviewDetailResponse, TransactionRiskExplanationResponse, existing AuditController/AuditQueryService, and existing financial/OTP/state/settlement services.

Retain existing response-field names even where new staff DTOs use a different explicit contract. Do not rename existing currencyCode fields to currency throughout the codebase.

17. Test strategy, execution gates, and expected-count discipline

17.1 Existing baseline

Evidence	Status
AccountControllerTest	18 passed in user rerun
AccountEndpointSecurityTest	User reported individual success; planned count 25
AccountServiceImplTest	10 in the focused group
SecurityAuthorizationContractTest	7 in the focused group
Combined account group	User screenshot confirms 60 passed
Full suite	User screenshot confirms 918 passed, 0 failures/errors/skips

These are historical results from this task, not a fresh execution during documentation creation. The old 870 report aggregate is not the current verification baseline.

17.2 Test placement

Test root: C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss

Proposed focused new tests:

Absolute path	Coverage
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/security/StaffReadAccessTest.java	Live status and exact-role helper
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/security/StaffReadEndpointSecurityTest.java	Filter chain/method security for new routes
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/controller/AdminAccountControllerTest.java	Input/output field contract
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/services/AdminAccountServiceImplTest.java	Filters, detail eligibility, DTO privacy
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/controller/CustomerProfileControllerTest.java	Own identity and safe profile
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/beans/PaymentCategoryTest.java	Enum/boundary semantics
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/controller/AuditEvidenceControllerTest.java	Auditor routes and request validation
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/services/AuditEvidenceServiceImplTest.java	Evidence and privacy rules
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/repository/ReportingReadRepositoryOracleIntegrationTest.java	Real Oracle view/aggregate mapping
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/controller/AdminOperationsControllerTest.java	Stats/failure contracts only
C:/Users/Aditya Rao/Downloads/Training/Project/SafePay/backend/src/test/java/com/ofss/services/AdminOperationsServiceImplTest.java	Operational query and mapping behavior

Extend existing related tests rather than duplicating them, especially:

- SecurityAuthorizationContractTest.

- AccountDaoIntegrationTest.
- AdminUserSecurityControllerTest / AdminUserSecurityServiceImplTest.
- UserDaoIntegrationTest / UserRoleDaoIntegrationTest / UserServiceImplTest.
- TransactionControllerTest / TransactionQueryServiceTest / TransactionDaoIntegrationTest.
- CreateTransactionRequestTest / TransactionEntityTest / TransactionResponseTest.
- TransactionCreationServiceTest and existing idempotency tests.
- RiskReviewControllerTest / RiskReviewServiceImplTest / RiskReviewResponseTest / RiskReviewRepositoryContractTest / RiskReviewServiceOracleIntegrationTest.
- LedgerDaoOracleIntegrationTest / RiskPolicyDaoIntegrationTest.
- Existing financial/authorization/cancellation/state/OTP/settlement regressions when category integration touches their fixtures.

Each existing test name is under the test root in its matching package. Confirm filenames and dependencies before editing; tests must prove behavior, not mirror implementation line by line.

17.3 Execution discipline

1. Perform static review and the permitted offline compile/test-compile.
2. Report compilation separately from runtime success.
3. Provide exact focused Maven goals and expected counts after tests are implemented.
4. User executes tests with their approved environment.
5. Inspect the first underlying error, not only repeated Spring context failures.
6. Use controlled transactional fixtures/approved cleanup patterns; do not repurpose or destroy V12 seed data.
7. Run the appropriately scoped regression group; final clean full-suite run closes the change request.
8. Count tests from the final selected test definitions/current run. Do not invent a number or sum stale Surefire reports from mixed executions.
9. Do not automatically run tests because compile permission exists.

Existing test configuration disables real mail connection checks and settlement processing; Maven already disables relevant scheduled production work and OTP delivery. Preserve these protections. Do not expose credentials in commands/logs or ask the user to paste secrets. No new code/tests/database commands are executed as part of creating this guide.

18. Critical compatibility decisions and approval checklist

Topic	Current disposition / required gate
Legacy category/purpose	Settled: null category, preserve purpose, no backfill
Threshold	Settled: strictly above INR 100000.00
Others note	Settled planning rule: reuse purpose, 1–140 trimmed characters
Account list/detail privacy	Settled: eight list fields including accountId; full number only individual staff detail
Auditor reservation figures	Settled: reservations/difference permitted, current/available balance omitted
New health feature	Removed
New category database objects	One enum and one nullable payment column; migration execution still requires approval
Strict new high-value request requirement	Required by the selected behavior; old high-value POST/replay compatibility consequence must be acknowledged at Phase 5
Existing legacy payment lifecycle	Must continue without requiring retroactive category
Date/search/stat/failure semantics	Concrete recommendations in this guide; included in phase approval, not silent defaults
Actual live migration/grant state	Verify through approved inspection before database-dependent execution
Backend call-site/DTO changes	Scoped additions; confirm constructor/replay compatibility in tests
Frontend selector/queue controls	Integration requirements only until actual frontend work is approved
Additional reporting index	Not preapproved; explain necessity based on evidence if encountered
Source-file remarks outside selected scope	Do not implement merely because present in an older note

A requirement of literal zero defects cannot be established by planning alone. Acceptance is based on the explicit contracts, compilation, negative-security tests, Oracle query/migration verification and user-run runtime gates.

19. Completion and continuation checklist

Before starting

- ☐ User approves the contract/first phase and its files.
- ☐ Refresh only relevant source and current workspace changes.
- ☐ Confirm no unrelated edits are included.
- ☐ Record phase-specific test expectations after test creation.

Before category deployment

- ☐ Explicit new migration approval and next-version confirmation.
- ☐ Existing rows retain null category and untouched purpose.
- ☐ New lower-value records cannot accept category.
- ☐ Existing high-value records remain processable.
- ☐ High-value POST/replay compatibility choice acknowledged.
- ☐ Flyway target and mapping deployment coordinated by the user.
- ☐ No financial/ledger/state mutation introduced.

Before final closure

- ☐ Every selected route has exact-role protection and bounded reads.
- ☐ No full account number in list/balance staff responses.
- ☐ No current/available balances in auditor reservation reconciliation.
- ☐ No new SYSTEM_ADMIN health functionality.
- ☐ Removed features have not re-entered work or pending lists.
- ☐ User-run focused and full-suite results recorded accurately.
- ☐ OpenAPI and final understanding/API-testing guides completed at the final stage.
- ☐ Implementation status distinguishes proposed, compiled, and runtime-verified work.

Continuation instruction: start with the next explicitly approved phase. Reading this guide does not authorize code, SQL, environment changes, application startup, tests, or later phases.